MediKiosk — Key Source Code

Offline-first AI patient case-taking for Indian hospital OPDs · Smart India Hackathon 2026, problem SIH26047 (All India Institute of Ayurveda, Ministry of Ayush)

Patients give their history **before** the consultation, by **voice or touch** in **Hindi or English**, at a kiosk or on their phone. Old paper reports are photographed and read by OCR. The doctor opens a structured summary in which every fact is labelled *patient said*, *from report*, *doctor verified* or *rejected*, and verifies it section by section. Everything runs on a local hospital server; the AI understands answers and drafts one narrative paragraph, but never asks its own questions, decides emergencies or diagnoses.

Part	Technology	Role
API server	Python 3.11 · FastAPI · PostgreSQL 16	Patients, queue, alerts, summary, FHIR outbox, audit
NLP service	IndicConformer ASR · Qwen2.5-3B (Ollama) · RapidOCR	Speech → text, answer extraction, OCR, summary
Web app	React 18 · TypeScript · Tailwind	Doctor, triage, desk, display board, admin
Android app	Kotlin · Jetpack Compose	Kiosk, hall station, patient app (Z Fold)
Alarm relay / mock HIS	Standalone FastAPI apps	Emergency path independent of the server; FHIR inbox

Verified: 483 automated tests pass; an end-to-end smoke test (register → 35 questions → OCR → AI summary → doctor approval → FHIR bundle delivered) passes on the live stack.

1. Red-flag rules engine

`server/app/services/redflags.py`

Product rule 3: emergencies are decided by clinician-written rules in content/redflags.yaml, never by the AI. One alert per session per rule; the WebSocket announcement is sent only after the database commit.

```
def _holds(cond: dict[str, Any], slots: dict[str, Any]) -> bool:
    if slots.get(cond.get("slot")) is None:
        return False
    v = slots[cond["slot"]]
    try:
        if "is" in cond:
            return v == cond["is"]
        if "in" in cond:
            return bool(set(v) & set(cond["in"])) if isinstance(v, list) else v in cond["in"]
        if "gte" in cond:
            return v >= cond["gte"]
        if "lte" in cond:
            return v <= cond["lte"]
    except TypeError:
        return False
    return False

def evaluate(slots: dict[str, Any]) -> list[dict[str, Any]]:
    fired = []
    for rule in get_rules():
        all_ok = all(_holds(c, slots) for c in rule.get("all") or [])
        any_conds = rule.get("any")
        any_ok = any(_holds(c, slots) for c in any_conds) if any_conds else True
        if all_ok and any_ok and (rule.get("all") or any_conds):
            fired.append(rule)
    return fired

def evaluate_and_alert(db: DB, session: Session, slots: dict[str, Any]) -> list[Alert]:
    created: list[Alert] = []
    for rule in evaluate(slots):
        # Once per session per rule: slots persist across answers, so re-raising after a nurse has
        # confirmed/downgraded would only add noise (deliberately stricter than "open/acknowledged").
        exists = db.exec(select(Alert.id).where(Alert.session_id == session.id, Alert.rule_id == rule["id"])).first()
        if exists:
            continue
        alert = Alert(
            kiosk_id=session.kiosk_id,
            session_id=session.id,
            level=rule["level"],
            rule_id=rule["id"],
            reason_text=rule["reason_text"],
            source=AlertSource.rule.value,
            status=AlertStatus.open.value,
        )
        db.add(alert)
        db.flush()
        audit.record(
            db,
            "system",
            "alert.create",
            "alert",
            alert.id,
            {"session_id": session.id, "rule_id": rule["id"], "level": rule["level"]},
        )
        _emit_after_commit(db, alert_public(alert))
        created.append(alert)
    return created
```

2. Queue selector

`server/app/services/queue.py`

Contract §10.2: emergency first, then normal patients who waited too long (aging), then priority rechecks (skipping ones reserved for another doctor), then the oldest normal patient.

```
def select_next(db: DB, department: str, doctor_id: str) -> Token | None:
    candidates = _today(db, department, (TokenStatus.waiting.value,))  # already ordered by arrival_seq
    for t in candidates:
        if t.lane == Lane.emergency:
            return t
    for t in candidates:
        if is_aged(t):
            return t
    for t in candidates:
        if t.lane == Lane.priority and t.preferred_doctor_id in (None, doctor_id):
            return t
    for t in candidates:
        if t.lane == Lane.normal:
            return t
    return None
```

3. Adaptive question graph

`server/app/services/graph.py`

Questions come only from YAML files (rule 2). The walk skips answered questions and follows branch rules over every answer so far, including triage answers; Ayurveda sessions continue into the AYUSH question set.

```
def resolve_next(question: dict[str, Any], slots: dict[str, Any]) -> str:
    nxt = question.get("next", "end")
    if isinstance(nxt, str):
        return nxt
    for rule in nxt:
        if "if" not in rule or condition_holds(rule["if"], slots):
            return rule["goto"]
    return "end"

def _walk(pathway: Pathway, answered: set[str], slots: dict[str, Any]) -> str | None:
    cur = pathway.start
    for _ in range(MAX_STEPS):
        if cur == "end" or cur not in pathway.questions:
            return None
        if cur not in answered:
            return cur
        cur = resolve_next(pathway.questions[cur], slots)
    raise ApiError(500, "graph_cycle", f"Question graph {pathway.id} has a cycle")

def next_question(db: DB, session: Session) -> tuple[dict[str, Any], dict[str, int]] | None:
    pathways = session_pathways(session)
    answered = {a.question_id for a in confident_answers(db, session.id)}
    slots = session_slots(db, session.id)
    for i, p in enumerate(pathways):
        qid = _walk(p, answered, slots)
        if qid is None:
            continue
        remaining = _remaining_from(p, qid, slots) + sum(_remaining_from(q, q.start, slots) for q in pathways[i + 1 :])
        done = sum(1 for q in pathways for a in answered if a in q.questions)
        return p.questions[qid], {"answered": done, "remaining_estimate": remaining}
    return None
```

4. Voice-answer grounding check

`nlp/app/extract.py`

Added after a live test where misheard "both sides" came back from the language model as a confident "right". A chosen option must actually be heard in the transcript, otherwise the patient is asked "Did you mean…?" with tap buttons.

```
def label_score(slot_name: str, value: str, text: str) -> float | None:
    label = choice_labels().get(slot_name, {}).get(value)
    if label is None:
        return None
    parts = [p.strip().lower() for p in label.split(",") if p.strip()] + [value.lower()]
    return max(fuzz.partial_ratio(p, text.lower()) for p in parts)

def ungrounded(slot: SlotSpec, value: Any, text: str, language: str = "hi") -> str | None:
    if value is None or slot.type not in ("enum", "list_enum"):
        return None
    if language == "hi" and not re.search(r"[\u0900-\u097F]", text):
        return None
    if slot.type == "enum":
        mine = label_score(slot.name, value, text)
        if mine is None:
            return None
        best = max((label_score(slot.name, v, text) or 0) for v in slot.enum or [value])
        if mine < SINGLE_MIN or mine < best:
            return f"{slot.name} '{value}' not heard in the answer (match {mine:.0f}, best {best:.0f})"
        return None
    weak = [v for v in value if (sc := label_score(slot.name, v, text)) is not None and sc < MULTI_MIN]
    return f"{slot.name} {weak} not heard in the answer" if weak else None
```

5. Critical strip of the doctor summary

`server/app/services/summary.py`

Rule 12: allergies are always shown, whatever their trust label; red flags and abnormal lab values are rendered by templates from structured data, not by the AI (rule 4).

```
def critical(db: DB, session: Session, facts: list[Fact]) -> dict[str, Any]:
    allergy_facts = [f for f in facts if f.section == Section.allergies]
    has_detail = any(f.key == "allergy_detail" for f in allergy_facts)
    allergies = []
    for f in allergy_facts:  # rule 12: always shown, whatever the trust
        text = _allergy_text(f, has_detail)
        if text:
            allergies.append({"fact_id": f.id, "text": text, "trust": f.trust})
    alerts = db.exec(
        select(Alert)
        .where(Alert.session_id == session.id, Alert.status != AlertStatus.downgraded)
        .order_by(col(Alert.created_at))
    ).all()
    red_flags = [{"alert_id": a.id, "text": a.reason_text, "level": a.level, "status": a.status} for a in alerts]
    abnormal = [
        {"fact_id": f.id, "text": f.text_en, "trust": f.trust}
        for f in facts
        if f.section == Section.labs
        and f.trust != Trust.rejected
        and str((f.value or {}).get("flag", "")).lower() in {"low", "high"}
    ]
    return {"allergies": allergies, "red_flags": red_flags, "abnormal_labs": abnormal}
```

6. Hash-chained audit log

`server/app/audit.py`

Rule 8: every create/update/verify is appended to a chain where each row hashes the previous one, so editing any past row is detected by verify_chain (shown live in the admin console).

```
def compute_hash(
    prev_hash: str, ts: datetime, actor: str, action: str, entity: str, entity_id: str | None, details: dict
) -> str:
    body = {
        "ts": iso_z(ts),
        "actor": actor,
        "action": action,
        "entity": entity,
        "entity_id": entity_id,
        "details": details,
    }
    return hashlib.sha256((prev_hash + canonical_json(body)).encode("utf-8")).hexdigest()

def record(
    db: Session, actor: str, action: str, entity: str, entity_id: str | None, details: dict[str, Any] | None = None
) -> AuditLog:
    _lock_chain(db)
    details = json.loads(canonical_json(details or {}))  # store exactly what was hashed
    last = db.exec(select(AuditLog).order_by(col(AuditLog.id).desc()).limit(1)).first()
    prev_hash = last.hash if last else GENESIS
    ts = utcnow()
    row = AuditLog(
        ts=ts,
        actor=actor,
        action=action,
        entity=entity,
        entity_id=entity_id,
        details=details,
        prev_hash=prev_hash,
        hash=compute_hash(prev_hash, ts, actor, action, entity, entity_id, details),
    )
    db.add(row)
    db.flush()
    return row

def verify_chain(db: Session) -> tuple[bool, int, int | None]:
    expected_prev = GENESIS
    checked = 0
    for row in db.exec(select(AuditLog).order_by(col(AuditLog.id))):
        checked += 1
        recomputed = compute_hash(row.prev_hash, row.ts, row.actor, row.action, row.entity, row.entity_id, row.details)
        if row.prev_hash != expected_prev or row.hash != recomputed:
            return False, checked, row.id
        expected_prev = row.hash
    return True, checked, None
```

7. Outbox worker: island mode + consent re-check

`server/app/services/outbox.py`

Rule 9: nothing leaves the hospital directly. FHIR bundles wait in the outbox; nothing is sent in island mode, consent is re-checked at send time, and failures retry with exponential backoff.

```
def consent_ok(session: DB, session_id: str) -> bool:
    c = session.exec(
        select(Consent).where(Consent.session_id == session_id).order_by(col(Consent.created_at).desc())
    ).first()
    return bool(c and c.share_hospital and c.revoked_at is None)
```

```python
def drain_once(session: DB, client: httpx.Client, now: datetime | None = None) -> dict[str, int]:
    counts = {"sent": 0, "retry": 0, "skipped": 0}
    if island_mode(session):
        return counts
    now = now or utcnow()
    due = [
        r
        for r in session.exec(
            select(Outbox).where(Outbox.status == OutboxStatus.pending).order_by(col(Outbox.created_at))
        )
        if as_utc(r.next_attempt_at) <= now
    ]
    for row in due:
        if island_mode(session):  # the switch can be flipped mid-drain
            break
        if not consent_ok(session, row.session_id):
            row.status = OutboxStatus.skipped_no_consent.value
            audit.record(session, "system", "outbox.skip", "outbox", row.id, {"reason": "no_consent"})
            counts["skipped"] += 1
        else:
            error = None
            try:
                res = client.post(
                    f"{settings.his_url}/fhir",
                    json=row.payload,
                    headers={"Idempotency-Key": row.idempotency_key},
                    timeout=SEND_TIMEOUT_SEC,
                )
                if not 200 <= res.status_code < 300:
                    error = f"HTTP {res.status_code}: {res.text[:200]}"
            except httpx.HTTPError as exc:
                error = f"{type(exc).__name__}: {exc}"[:300]
            if error is None:
                row.status, row.sent_at, row.last_error = OutboxStatus.sent.value, utcnow(), None
                audit.record(session, "system", "outbox.sent", "outbox", row.id, {"key": row.idempotency_key})
                counts["sent"] += 1
            else:
                row.attempts += 1
                row.last_error = error
                row.next_attempt_at = utcnow() + backoff(row.attempts)
                audit.record(
                    session,
                    "system",
                    "outbox.retry",
                    "outbox",
                    row.id,
                    {"attempts": row.attempts, "error": error[:120]},
                )
                counts["retry"] += 1
        session.add(row)
        session.commit()
    return counts
```

8. Per-patient encryption and erasure

`server/app/services/crypto.py`

Contract §10.8: AES-256-GCM with one data key per patient, wrapped by a master key. Erasure (two admins) destroys the key, so every stored report and voice clip becomes unreadable.

```python
def seal(key: bytes, plaintext: bytes, aad: bytes) -> bytes:
    nonce = os.urandom(NONCE)
    return nonce + AESGCM(key).encrypt(nonce, plaintext, aad)

def open_sealed(key: bytes, blob: bytes, aad: bytes) -> bytes:
    return AESGCM(key).decrypt(blob[:NONCE], blob[NONCE:], aad)

def data_key(patient_id: str, create: bool = True) -> bytes:
    with DB(app_db.engine) as db:
        row = db.get(PatientKey, patient_id)
        if row is None:
            if not create:
                raise _erased()
            dek = os.urandom(32)
            nonce = os.urandom(NONCE)
            wrapped = nonce + _master().encrypt(nonce, dek, patient_id.encode())
            db.add(PatientKey(patient_id=patient_id, wrapped_key=base64.b64encode(wrapped).decode()))
            try:
                db.commit()
                return dek
            except IntegrityError:  # another request created it first — use theirs
                db.rollback()
                row = db.get(PatientKey, patient_id)
        return _unwrap(row)

def destroy_key(db: DB, patient_id: str) -> PatientKey:
    row = db.get(PatientKey, patient_id) or PatientKey(patient_id=patient_id)
    row.wrapped_key = None
    row.destroyed_at = row.destroyed_at or utcnow()
    db.add(row)
    return row
```